Chapitre **20**

Tas de Fibonacci

Au chapitre 19, nous avons vu comment les tas binomiaux pouvaient supporter en temps $O(\lg n)$, dans le cas le plus d favorable, les op rations de tas fusionnable INS RER, MINIMUM, EXTRAIRE-MIN et UNION, plus les op rations DIMINUER-CL et SUPPRIMER. Dans ce chapitre, nous tudierons les tas de Fibonacci, qui supportent les m mes op rations mais avec l'avantage suppl mentaire que les op rations qui ne n cessitent pas la suppression d'un l ment s'ex cutent avec un temps amorti $O(1)$.

D'un point de vue th orique, les tas de Fibonacci se r v lent particuli rement adapt s quand le nombre d'op rations EXTRAIRE-MIN et SUPPRIMER est petit par rapport aux autres op rations. Cette situation se produit dans de nombreuses applications. Par exemple, certains algorithmes de graphes sont susceptibles d'appeler DIMINUER-CL une fois pour chaque arc. Pour les graphes denses, qui ont de nombreux arcs, le temps amorti $O(1)$ de chaque appel DIMINUER-CL am liore grandement le temps $\Theta(\lg n)$ du cas le plus d favorable des tas binaires ou binomiaux. Certains algorithmes rapides pour des probl mes comme le calcul d'arbres couvrants minimaux (chapitre 23) et la recherche des plus courts chemins origine unique (chapitre 24) s'appuient fondamentalement sur des tas de Fibonacci.

D'un point de vue pratique, toutefois, les facteurs constants et la programmation complexe des tas de Fibonacci les rendent moins int ressants que les tas binaires (ou k-aires) ordinaires pour la plupart des applications. Les tas de Fibonacci sont donc surtout int ressants d'un point de vue th orique. Si l'on inventait une structure de donn es beaucoup plus simple ayant les m mes bornes temporelles amorties que les tas de Fibonacci, elle serait d'un grand int r t pratique et non plus seulement th orique.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

l'instar d'un tas binomial, un tas de Fibonacci est une collection d'arbres. Les tas de Fibonacci sont, en fait, vaguement inspir s des tas binomiaux. Si les proc dures DIMINUER-CL et SUPPRIMER ne sont jamais invoqu es dans un tas de Fibonacci, alors chaque arbre du tas ressemble un arbre binomial. Les tas de Fibonacci ont une structure plus souple que les tas binomiaux, cependant, ce qui donne de meilleures bornes temporelles asymptotiques. Le travail de mise jour de la structure peut tre effectu en diff r , si cela s'av re plus commode.

Comme les tables dynamiques de la section 17.4, les tas de Fibonacci offrent un bon exemple de structure de donn e con ue en ayant l'esprit une analyse amortie. La compr hension intuitive et l'analyse des op rations de tas de Fibonacci que nous verrons dans ce chapitre se basent largement sur la m thode du potentiel vue la section 17.3.

Ce chapitre suppose acquis les concepts du chapitre 19 sur les tas binomiaux. Les op rations utilis es ici sont sp cifi es dans ce dernier chapitre, de m me que le tableau de la figure 19.1,qui r sume les bornes de temps d'ex cution des op rations de tas binaire, de tas binomial et de tas de Fibonacci. Notre pr sentation de la structure de tas de Fibonacci s'appuie sur la pr sentation de la structure de tas binomial, et certaines des op rations effectu es sur les tas de Fibonacci sont semblables celles faites sur les tas binomiaux.

Comme les tas binomiaux, les tas de Fibonacci ne sont pas pr vus pour supporter efficacement l'op ration RECHERCHER ; les op rations qui font r f rence un nœud ont donc besoin d'avoir dans leurs param tres d'entr e un pointeur vers ce nœud. Quand on utilise un tas de Fibonacci dans une application, on stocke souvent dans chaque l ment du tas un handle r f ren ant l'objet d'application correspondant et l'on stocke dans chaque objet d'application un handle r f ren ant l' l ment de tas de Fibonacci correspondant

La section 20.1 d finit les tas de Fibonacci, tudie leur repr sentation et pr sente la fonction potentiel utilis e pour leur analyse amortie. La section 20.2 montre comment impl menter les op rations de tas fusionnable et atteindre les bornes amorties donn es la figure 19.1.Les deux derni res op rations, DIMINUER-CL et SUPPRIMER, sont pr sent es la section 20.3. Enfin, la section 20.4 termine en d taillant une partie fondamentale de l'analyse et explique l'origine trange de la structure de donn es.

20.1 STRUCTURE DES TAS DE FIBONACCI

Comme les tas binomiaux, un ***tas de Fibonacci*** est un ensemble d'arbres ordonn s selon une structure de tas min. Toutefois, les arbres d'un tas de Fibonacci ne sont pas obligatoirement des arbres binomiaux. La figure 20.1(a) donne un exemple de tas de Fibonacci.

Scanned with CamScanner

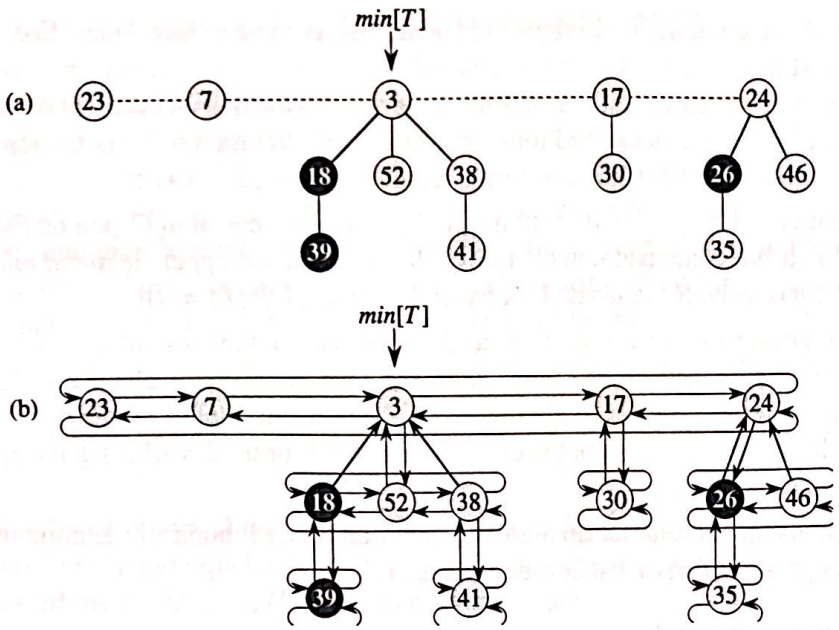


Figure 20.1 **(a)** Un tas de Fibonacci constitu de cinq arbres ordonn s en tas min et de 14 nœuds. La ligne pointill e repr sente la liste des racines. Le nœud minimal du tas est le nœud contenant la cl 3. Les trois nœuds marqu s sont en noir. Le potentiel de ce tas de Fibonacci particulier est $5+2\cdot3=11$. **(b)** Une repr sentation plus compl te, faisant appara tre les pointeurs *p* (fl ches vers le haut), *enfant* (fl ches vers le bas) et *gauche* et *droite* (fl ches horizontales). Ces d tails sont omis pour les autres figures de ce chapitre, puisque toutes les informations pr sentes ici peuvent tre d termin es partir de celles de la partie (a).

Contrairement aux arbres contenus dans les tas binomiaux, qui sont tri s, ceux contenus dans les tas de Fibonacci sont enracin s mais non tri s. Comme le montre la figure 20.1(b), chaque nœud x contient un pointeur $p[x]$ sur son parent et un pointeur $enfant[x]$ sur l'un quelconque de ses enfant. Les enfant de x sont reli s dans une liste doublement cha n e, que nous appellerons la ***liste des enfants*** de x. Chaque enfant y d'une liste de enfant poss de les pointeurs $gauche[y]$ et $droite[y]$, qui pointent respectivement sur les fr res de gauche et de droite de y. Si le nœud y est un enfant unique, alors $gauche[y] = droite[y] = y$. L'ordre dans lequel apparaissent les fr res dans la liste des enfants est arbitraire.

Les listes circulaires doublement cha n es (voir section 10.2) ont deux avantages qui serviront pour le traitement des tas de Fibonacci. Primo, un nœud peut tre supprim en temps $O(1)$. Ensuite, tant donn es deux listes de ce type, on peut les concat ner (les « greffer » l'une sur l'autre) $O(1)$ pour obtenir une seule liste circulaire doublement cha n e en temps $O(1)$. Lorsque nous d crirons les op rations de tas de Fibonacci, nous nous y r f rerons de mani re informelle, en laissant au lecteur les d tails de leur impl mentation.

Chaque nœud a deux autres champs qui nous seront utiles. Le nombre d'enfants contenus dans la liste des enfant d'un nœud x est conserv dans $degr[x]$. Le champ

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

valeur bool enne *marqu* [x] indique si le nœud x a perdu un enfant depuis la derni re fois o x est devenu enfant d'un autre nœud. Les nœuds nouvellement cr s ne sont pas marqu s et un nœud x est « d marqu » chaque fois qu'il devient enfant d'un autre nœud. Jusqu' ce que nous tudiions l'op ration DIMINUER-CL la section 20.3, nous nous contenterons d'initialiser tous les champs *marqu* FAUX.

On peut acc der un tas de Fibonacci T par un pointeur *min*[T] pointant vers la racine d'un arbre contenant une cl minimale ; ce nœud est appel le ***nœud minimal*** du tas de Fibonacci. Si un tas de Fibonacci T est vide, *min*[T] = NIL.

Les racines de tous les arbres d'un tas de Fibonacci sont reli es entre elles, l'aide de leurs pointeurs *gauche* et *droite*, dans une liste doublement cha n e circulaire, appel e ***liste des racines*** du tas de Fibonacci. Le pointeur *min*[T] pointe donc sur un nœud de la liste des racines dont la cl est minimale. L'ordre des arbres dans une liste des racines est arbitraire.

Nous nous appuierons sur un autre attribut d'un tas de Fibonacci : le nombre total de nœuds pr sents dans T est conserv dans n[T].

a) Fonction potentiel

Comme nous l'avons dit, nous utiliserons la m thode du potentiel de la section 17.3 pour analyser les performances des op rations de tas de Fibonacci. Pour un tas de Fibonacci T, on repr sente par $a(T)$ le nombre d'arbres contenus dans la liste des racines de T et par $m(T)$ le nombre de nœuds marqu s dans T. Le potentiel du tas de Fibonacci T est alors d fini par

$$\Phi(T) = a(T) + 2\,m(T)\,. \tag{20.1}$$

Par exemple, le potentiel du tas de Fibonacci montr la figure 20.1 est $5 + 2 \cdot 3 = 11$. Le potentiel d'un ensemble de tas de Fibonacci est la somme des potentiels des tas de Fibonacci qui le constituent. On supposera qu'une unit de potentiel permet de payer une quantit de travail constante, la constante tant suffisamment grande pour couvrir le co t de n'importe quel travail partiel sp cifique temps constant que nous pourrions avoir effectuer.

On suppose qu'une application utilisant les tas de Fibonacci commence sans tas. Le potentiel initial vaut donc 0 et, d'apr s l' quation (20.1), le potentiel reste ensuite toujours positif ou nul. D'apr s l' quation (17.3), un majorant du co t total amorti est donc un majorant du co t total r el de la s quence d'op rations.

b) Degr maximal

Les analyses amorties que nous effectuerons dans les sections suivantes de ce chapitre supposent qu'il existe un majorant connu $D(n)$ pour le degr maximal d'un nœud quelconque d'un tas de Fibonacci n nœuds. L'exercice 20.2.3 montrera que, quand les op rations de tas fusionnables sont les seules tre support es, alors $D(n) \leqslant \lfloor \lg n \rfloor$. A la section 20.3, nous montrerons que, quand DIMINUER-CL et SUPPRIMER sont galement support es, alors $D(n) = O(\lg n)$.

Scanned with CamScanner

20.2 OP RATIONS SUR LES TAS FUSIONNABLES

Dans cette section, on d crit et analyse les op rations sur les tas fusionnables ba- s s sur des tas de Fibonacci. Si les op rations CR ER-TAS, INS RER, MINIMUM, EXTRAIRE-MIN et UNION sont les seules prendre en compte, alors chaque tas de Fibonacci est tout simplement une collection d'arbres binomiaux « non tri s ». Un ***arbre binomial non tri*** ressemble un arbre binomial et il est, lui aussi, d fini r cursivement. L'arbre binomial non tri U_0 est constitu d'un nœud unique et un arbre binomial non tri U_k est constitu de deux arbres binomiaux non tri s U_{k-1} pour lequel la racine de l'un devient un enfant *quelconque* de la racine de l'autre. Le lemme 19.1, qui donne les propri t s des arbres binomiaux, est galement valable pour les arbres binomiaux non tri s, mais avec une l g re variation dans la propri t 4 (voir exercice 20.2.2) :

4′. Pour l'arbre binomial non tri U_k, la racine a le degr k, qui est plus grand que celui de n'importe quel autre nœud. Les enfants de la racine sont les racines des sous-arbres $U_0, U_1, \ldots, U_{k-1}$ dans un certain ordre.

Donc, si un tas de Fibonacci n nœuds est une collection d'arbres binomiaux non tri s, alors $D(n) = \lg n$.

Pour les op rations de tas fusionnable sur les tas de Fibonacci, le principe est de diff rer le plus longtemps possible le travail effectuer. On doit faire un compromis concernant les performances des impl mentations des diff rentes op rations. Si le nombre d'arbres d'un tas de Fibonacci est petit, on peut rapidement d terminer le- quel des nœuds restants doit devenir le nouveau nœud minimal lors d'une op ration EXTRAIRE-MIN. Cependant, comme nous l'avons vu l'exercice 19.2.10 pour les tas binomiaux, garantir que le nombre d'arbres est petit n'est pas gratuit ; l'inser- tion d'un nœud dans un tas binomial ou l'union de deux tas binomiaux peut prendre jusqu' $\Omega(\lg n)$. Comme nous le verrons, on ne tente pas de consolider les arbres contenus dans un tas de Fibonacci lorsqu'on ins re un nouveau nœud ou qu'on r unit deux tas. On r serve la consolidation pour l'op ration EXTRAIRE-MIN, c'est- -dire le moment o l'on a r ellement besoin de trouver le nouveau nœud minimal.

c) Cr ation d'un nouveau tas de Fibonacci

Pour cr er un tas de Fibonacci vide, la proc dure CR ER-TAS-FIB alloue et retourne l'objet T, o $n[T] = 0$ et $min[T] =$ NIL ; T ne contient aucun arbre. Comme $a(T) = 0$ et $m(T) = 0$, le potentiel du tas de Fibonacci vide est $\Phi(T) = 0$. Le co t amorti de CR ER-TAS-FIB est donc gal son co t r el $O(1)$.

d) Insertion d'un nœud

La proc dure suivante ins re un nœud x dans un tas de Fibonacci T, en supposant que le nœud a d j t allou et que $cl[x]$ n'est pas vide.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

```
INS RER-TAS-FIB(T, x)
 1  degr [x] ← 0
 2  p[x] ← NIL
 3  enfant[x] ← NIL
 4  gauche[x] ← x
 5  droite[x] ← x
 6  marqu [x] ← FAUX
 7  concat ner liste de racines contenant x et liste de racines T
 8  si min[T] = NIL ou cl [x] < cl [min[T]]
 9     alors min[T] ← x
10  n[T] ← n[T] + 1
```

Apr s initialisation par les lignes 1–6 des champs du nœud x, cr ant ainsi sa propre liste circulaire doublement cha n e, la ligne 7 ajoute x la liste des racines de T dans un temps $O(1)$. Ainsi, le nœud x devient un arbre ordonn de type tas min contenant un seul nœud, et donc un arbre binomial non ordonn , du tas de Fibonacci. Il n'a aucun enfant et n'est pas marqu . Les lignes 8–9 mettent ensuite jour le pointeur vers le nœud minimal du tas de Fibonacci T, si n cessaire. Enfin, la ligne 10 incr mente $n[T]$ pour prendre en compte l'addition du nouveau nœud. La figure 20.2 montre l'insertion d'un nœud de cl 21 dans le tas de Fibonacci de la figure 20.1.

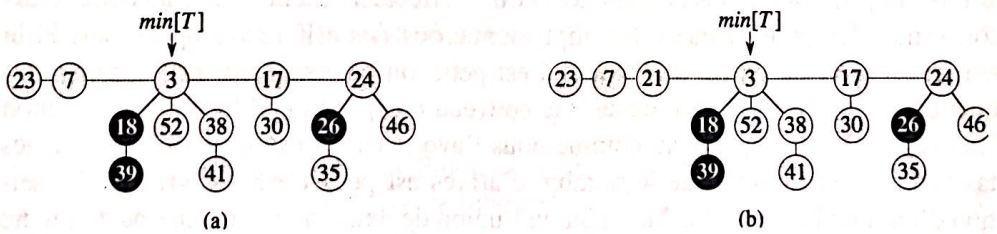


Figure 20.2 Insertion d'un nœud dans un tas de Fibonacci. **(a)** Un tas de Fibonacci T. **(b)** Le tas T apr s insertion du nœud de cl 21. Le nœud devient son propre arbre ordonn de type tas min et est ensuite ajout la liste des racines, ce qui en fait le fr re gauche de la racine.

Contrairement la proc dure INS RER-TAS-BINOMIAL, INS RER-TAS-FIB n'essaye pas de consolider les arbres l'int rieur du tas de Fibonacci. S'il y a k op rations INS RER-TAS-FIB cons cutives, alors k arbres un nœud sont ajout s la liste des racines.

Pour d terminer le co t amorti de INS RER-TAS-FIB, soit T le tas de Fibonacci en entr e et T' le tas de Fibonacci r sultant. On a $a(T') = a(T) + 1$ et $m(T') = m(T)$, et l'augmentation de potentiel est

$$((a(T) + 1) + 2\,m(T)) - (a(T) + 2\,m(T)) = 1\,.$$

Comme le co t r el est $O(1)$, le co t amorti est $O(1) + 1 = O(1)$.

Scanned with CamScanner

e) Recherche du nœud minimal

Le nœud minimal dans un tas de Fibonacci T est donn par le pointeur $min[T]$, ce qui permet de le retrouver dans un temps $O(1)$. Comme le potentiel de T n'est pas modifi , le co t amorti de cette op ration est gal son co t r el $O(1)$.

f) Union de deux tas de Fibonacci

La proc dure suivante r unit les tas de Fibonacci T_1 et T_2, d truisant T_1 et T_2 ce faisant. Elle se contente de concat ner les listes de racines de T_1 et T_2, puis elle d termine le nouveau nœud minimal.

```
UNION-TAS-FIB(T1, T2)
1  T ← CR ER-TAS-FIB()
2  min[T] ← min[T1]
3  concat ner liste de racines de T2 celle de T
4  si (min[T1] = NIL) ou (min[T2] ≠ NIL et cl [min[T2]] < cl [min[T1]])
5     alors min[T] ← min[T2]
6  n[T] ← n[T1] + n[T2]
7  lib rer objets T1 et T2
8  retourner T
```

Les lignes 1–3 concat nent les listes de racines de T_1 et T_2 en une nouvelle liste T. Les lignes 2, 4 et 5 initialisent le nœud minimal de T et la ligne 6 affecte $n[T]$ le nombre total de nœuds. Les objets T_1 et T_2 sont lib r s la ligne 7, et la ligne 8 retourne le tas de Fibonacci T final. Comme pour la proc dure INS RER-TAS-FIB aucune consolidation des arbres n'intervient.

Le changement de potentiel est

$$\begin{aligned}\Phi(T) - (\Phi(T_1) + \Phi(T_2)) &= (a(T) + 2\,m(T)) - ((a(T_1) + 2\,m(T_1)) + (a(T_2) + 2\,m(T_2))) \\ &= 0\,,\end{aligned}$$

car $a(T) = a(T_1) + a(T_2)$ et $m(T) = m(T_1) + m(T_2)$. Le co t amorti de UNION-TAS-FIB est donc gal son co t r el $O(1)$.

g) Extraction du nœud minimal

Le processus d'extraction du nœud minimal est l'op ration la plus compliqu e parmi celles qui sont pr sent es dans cette section. C'est galement cet endroit que survient le travail de consolidation diff r e des arbres figurant dans la liste de racines. Le pseudo code suivant extrait le nœud minimal. On suppose par commodit que, lorsqu'un nœud est supprim d'une liste cha n e, les pointeurs restant dans la liste sont mis jour mais que ceux du nœud extrait restent inchang s. On fait aussi appel la proc dure auxiliaire CONSOLIDER, qui sera pr sent e bient t.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

```
EXTRAIRE-MIN-TAS-FIB(T)
 1  z ← min[T]
 2  si z ≠ NIL
 3     alors pour chaque enfant x de z
 4              faire ajouter x  la liste de racines de T
 5                   p[x] ← NIL
 6           supprimer z de la liste de racines de T
 7           si z = droite[z]
 8             alors min[T] ← NIL
 9             sinon min[T] ← droite[z]
10                   CONSOLIDER(T)
11           n[T] ← n[T] − 1
12  retourner z
```

Comme le montre la figure 20.3, EXTRAIRE-MIN-TAS-FIB commence par transformer en racine chaque enfant du nœud minimal et par supprimer le nœud minimal de la liste de racines. Puis, elle consolide la liste des racines en reliant entre elles les racines de degr gal, jusqu' ce qu'il n'existe plus qu'une racine, au plus, pour chaque degr .

On commence en ligne 1 par sauvegarder un pointeur z sur le nœud minimal ; ce pointeur est retourn la fin de la proc dure. Si z = NIL, alors le tas de Fibonacci T est d j vide et c'est termin . Sinon, comme pour la proc dure EXTRAIRE-MIN-TAS-BINOMIAL, on supprime le nœud z de T en transformant en racines tous les enfant de z aux lignes 3–5 (ils sont plac s dans la liste des racines) et en supprimant z de la liste des racines la ligne 6. Si z = *droite*[z] apr s la ligne 6, alors z tait le seul nœud de la liste de racines et il n'avait pas de enfant ; il ne reste donc plus qu' vider la t te du tas de Fibonacci la ligne 8, avant de retourner z. Dans le cas contraire, on fait en sorte que le pointeur *min*[T] pointe vers un nœud de la liste de racines autre que z (dans ce cas *droite*[z]), qui ne va pas forc ment tre le nouveau nœud minimal quand EXTRAIRE-MIN-TAS-FIB aura fini. La figure 20.3(b) montre le tas de Fibonacci de la figure 20.3(a) apr s ex cution de la ligne 9.

L' tape suivante, pendant laquelle on r duit le nombre d'arbres du tas de Fibonacci, est la ***consolidation*** de la liste des racines de T ; cela est effectu par l'appel CONSOLIDER(T). Consolider la liste des racines revient ex cuter de mani re r p t e les tapes suivantes, jusqu' ce que toutes les racines de la liste aient une valeur *degr * distincte.

1) Trouver dans la liste des racines deux racines x et y ayant le m me degr et pour lesquelles $cl\ [x] \leqslant cl\ [y]$.
2) ***Relier*** y x : supprimer y de la liste des racines et faire de y un enfant de x. Cette op ration est prise en charge par la proc dure RELIER-TAS-FIB. Le champ *degr* [x] est incr ment et la marque ventuellement pr sente sur y est effac e.

Scanned with CamScanner

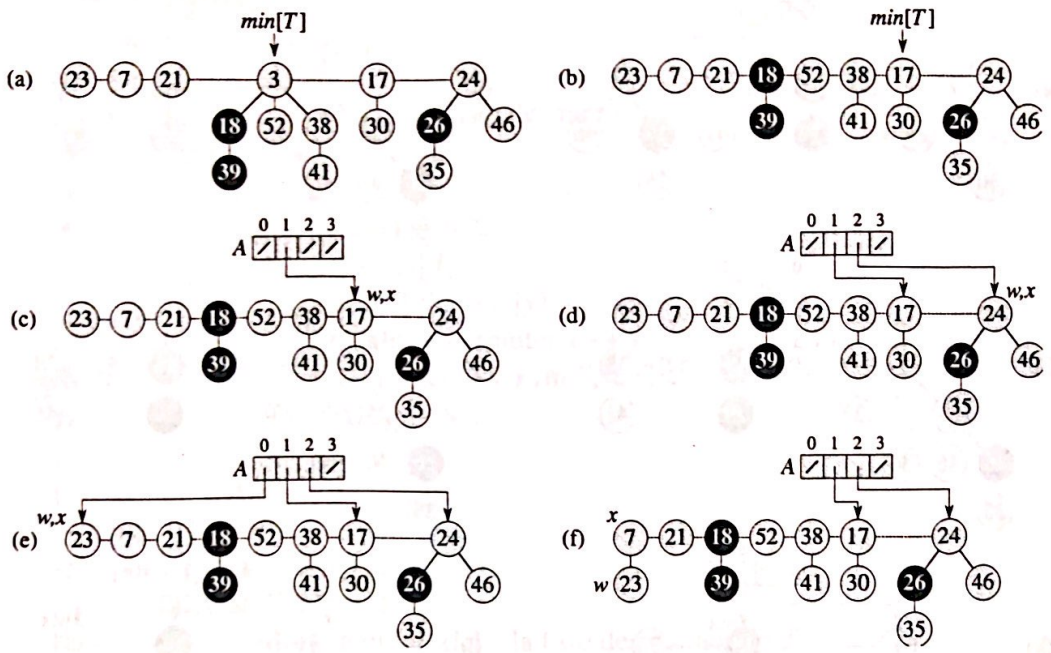
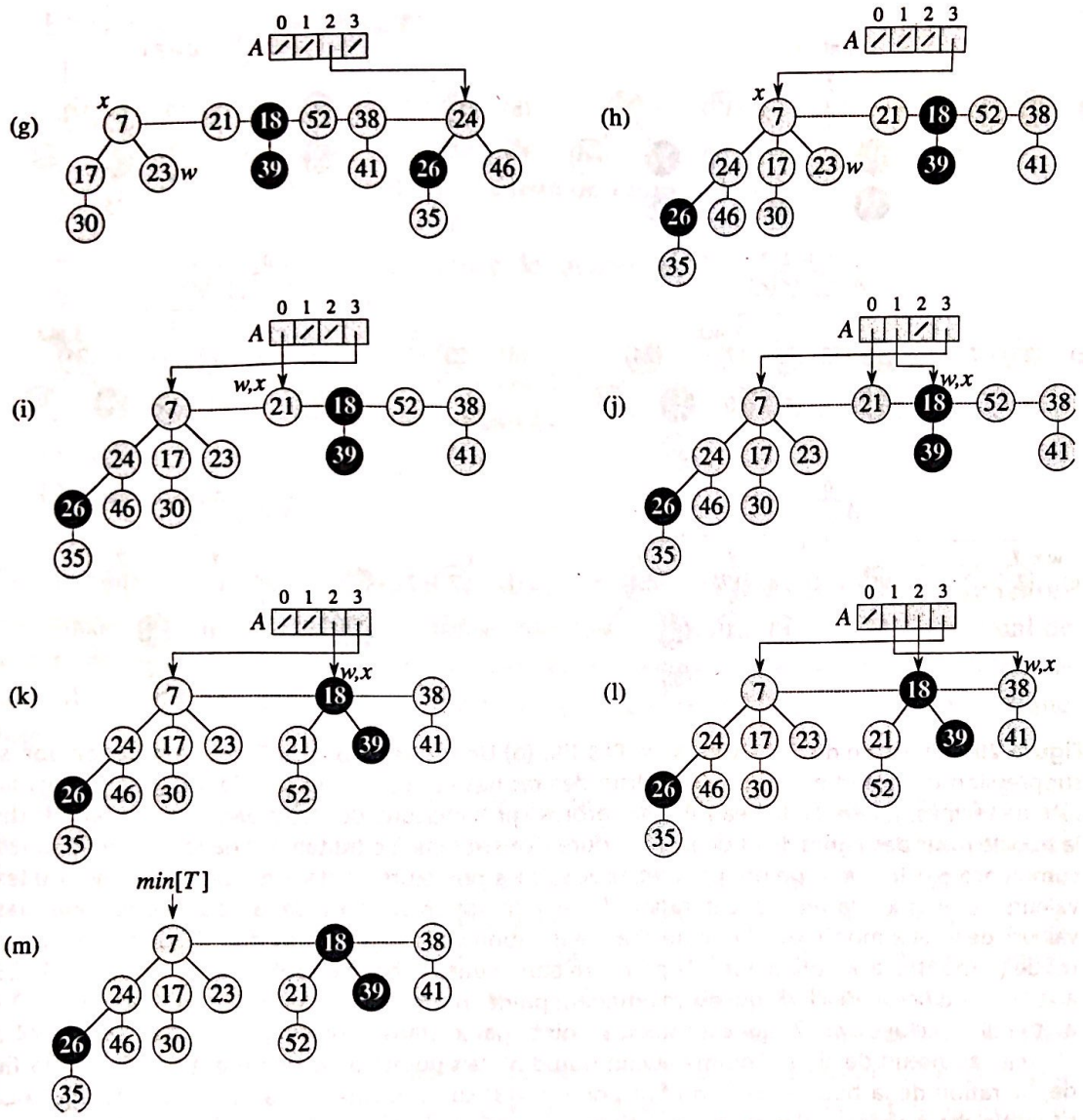


Figure 20.3 L'action de EXTRAIRE-MIN-TAS-FIB.**(a)** Un tas de Fibonacci T. **(b)** La situation apr s suppression du nœud minimal z de la liste des racines et apr s insertion de ses enfant dans la liste de racines. **(c)–(e)** Le tableau A et les arbres apr s chacune des trois premi res it rations de la boucle **pour** des lignes 3–13 de la proc dure CONSOLIDER. Le traitement de la liste des racines commence par le nœud point par *min*[T] et suit les pointeurs *droite*. Chaque partie montre les valeurs de w et x la fin d'une it ration. **(f)–(h)** L'it ration suivante de la boucle **pour**, avec les valeurs de w et x montr es la fin de chaque it ration de la boucle **tant que** des lignes 6–12. La partie (f) montre la situation apr s la premi re entr e dans la boucle **tant que**. Le nœud de cl 23 a t reli au nœud de cl 7, qui est maintenant point par x. Dans la partie (g), le nœud de cl 17 a t reli celui de cl 7, qui est toujours point par x. Dans la partie (h), le nœud de cl 24 a t reli au nœud de cl 7. Comme aucun nœud n' tait point pr c demment par $A[3]$, la fin de l'it ration de la boucle **pour**, on fait pointer $A[3]$ sur la racine de l'arbre r sultant. **(i)–(l)** La situation apr s chacune des quatre it rations suivantes de la boucle **pour**. **(m)** Le tas de Fibonacci T apr s reconstruction de la liste de racines partir du tableau A et d termination du nouveau pointeur *min*[T].

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner



La proc dure CONSOLIDER utilise un tableau auxiliaire $A[0\,..\,D(n[T])]$; si $A[i] = y$, alors y est actuellement une racine pour laquelle $degr\ [y] = i$.

Scanned with CamScanner

```
CONSOLIDER(T)
 1  pour i ← 0  D(n[T])
 2     faire A[i] ← NIL
 3  pour chaque nœud w de la liste des racines de T
 4     faire x ← w
 5           d ← degr [x]
 6           tant que A[d] ≠ NIL
 7              faire y ← A[d]
 8                    si cl [x] > cl [y]
 9                       alors permuter x ↔ y
10                    RELIER-TAS-FIB(T, y, x)
11                    A[d] ← NIL
12                    d ← d + 1
13           A[d] ← x
14  min[T] ← NIL
15  pour i ← 0  D(n[T])
16     faire si A[i] ≠ NIL
17           alors ajouter A[i]  la liste des racines de T
18                 si min[T] = NIL ou cl [A[i]] < cl [min[T]]
19                    alors min[T] ← A[i]
```

```
RELIER-TAS-FIB(T, y, x)
1  supprimer y de la liste des racines de T
2  faire de y un enfant de x, incr menter degr [x]
3  marqu [y] ← FAUX
```

Dans le d tail, la proc dure CONSOLIDER fonctionne de la mani re suivante. Les lignes 1–2 initialisent A en affectant chaque l ment NIL. La boucle **pour** des lignes 3–13 traite chaque racine w de la liste de racines. Apr s traitement de chaque racine w, celle-ci se retrouve dans un arbre enracin en un certain nœud x, qui peut ou non tre identique w. Parmi les racines trait es, aucune autre n'aura le m me degr que x, et donc on fait pointer l' l ment $A[degr\ [x]]$ vers x. Quand cette boucle **pour** se terminera, il restera au plus une racine pour chaque degr , et le tableau A pointera alors vers chaque racine restante.

La boucle **tant que** des lignes 6–12 relie, de mani re r p t e, la racine x de l'arbre contenant le nœud w un autre arbre dont la racine a le m me degr que x, et ce jusqu' ce qu'il n'y ait plus aucune racine qui ait le m me degr . Cette boucle **tant que** conserve l'invariant que voici : Au d but de chaque it ration de la boucle **tant que**, $d = degr\ [x]$.

Nous utiliserons ainsi cet invariant :

Initialisation : La ligne 5 assure que l'invariant est vrai la premi re fois que l'on entre dans la boucle.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

Conservation : chaque it ration de la boucle **tant que**, $A[d]$ pointe vers une certaine racine y. Comme $d = degr\ [x] = degr\ [y]$, on doit relier x et y. Celui de x et y qui a la plus petite cl devient le parent de l'autre dans le cadre de l'op ration de liaison ; donc, les lignes 8–9 permutent, si n cessaire, les pointeurs pointant vers x et y. Ensuite, on relie y x *via* l'appel RELIER-TAS-FIB(T, y, x) en ligne 10. Cet appel incr mente $degr\ [x]$ mais laisse $degr\ [y]$ d. Comme le nœud y n'est plus une racine, le pointeur pointant vers lui est supprim du tableau A en ligne 11. Comme l'appel de RELIER-TAS-FIB incr mente la valeur de $degr\ [x]$, la ligne 12 restaure donc l'invariant selon lequel $d = degr\ [x]$.

Terminaison : On r p te la boucle **tant que** jusqu' ce que $A[d]$ = NIL, auquel cas il n'y a aucune autre racine qui ait le m me degr que x.

Apr s la fin de la boucle **tant que**, on affecte $A[d]$ la valeur x en ligne 13 et l'on fait l'it ration suivante de la boucle **pour**.

Les figures 20.3(c)–(e) montrent le tableau A et les arbres r sultants apr s les trois premi res it rations de la boucle **pour** des lignes 3–13. Lors de l'it ration suivante de la boucle **pour**, il y a cr ation de trois liaisons ; leurs r sultats sont montr s sur les figures 20.3(f)–(h). Les figures 20.3(i)–(l) montrent le r sultat des quatre it rations suivantes de la boucle **pour**.

Tout ce qui reste faire maintenant, c'est le nettoyage. Une fois que la boucle **pour** des lignes 3–13 a pris fin, la ligne 14 vide la liste de racines et les lignes 15–19 reconstruisent cette liste partir du tableau A. Le tas de Fibonacci r sultant est montr la figure 20.3(m). Apr s consolidation de la liste de racines, EXTRAIRE-MIN-TAS-FIB se termine en d cr mentant $n[T]$ en ligne 11 et en retournant un pointeur vers le nœud supprim z en ligne 12.

Observez que, si tous les arbres du tas de Fibonacci sont des arbres binomiaux non ordonn s avant l'ex cution de EXTRAIRE-MIN-TAS-FIB, ils le restent apr s. Les arbres peuvent tre modifi s de deux mani res. Primo, aux lignes 3–5 de EXTRAIRE-MIN-TAS-FIB, chaque enfant x de la racine z devient une racine. D'apr s l'exercice 20.2.2, chaque nouvel arbre est lui-m me un arbre binomial non ordonn . Secundo, les arbres ne sont reli s par RELIER-TAS-FIB que s'ils ont le m me degr . Comme tous les arbres sont des arbres binomiaux non ordonn s avant la liaison, deux arbres dont les racines ont chacune k enfants doivent avoir la structure de U_k. L'arbre r sultant a donc la structure de U_{k+1}.

On peut maintenant montrer que le co t amorti de l'extraction du nœud minimal d'un tas de Fibonacci n nœuds est $O(D(n))$. Soit T le tas de Fibonacci juste avant l'op ration EXTRAIRE-MIN-TAS-FIB.

Voici comment on peut valuer le co t r el de l'extraction du nœud minimal. Il y a une contribution $O(D(n))$ qui vient de ce qu'il y a au plus $D(n)$ enfants du nœud minimal qui sont trait s dans EXTRAIRE-MIN-TAS-FIB et du travail fait sur les lignes 1–2 et 14–19 de CONSOLIDER. Reste analyser la contribution de la boucle **pour** des lignes 3–13. La taille de la liste de racines, lors de l'appel de CONSOLIDER, est

Scanned with CamScanner

d'au plus $D(n) + t(T) - 1$, vu que la liste se compose des $t(T)$ nœuds originel de la liste, moins le nœud racine extrait, plus les enfants du nœud extrait qui sont au plus $D(n)$. chaque passage dans la boucle **tant que** des lignes 6–12, l'une des racines est reli e une autre ; donc, le volume total de travail effectu dans la boucle **pour** est au plus proportionnel $D(n) + t(T)$. Par cons quent, le travail r el total induit par l'extraction du nœud minimal est $O(D(n) + t(T))$.

Le potentiel avant extraction du nœud minimal est $t(T) + 2\,m(T)$; le potentiel apr s extraction est d'au plus $(D(n) + 1) + 2\,m(T)$, car il reste au plus $D(n) + 1$ racines et aucun nœud n'est marqu pendant l'op ration. Le co t amorti est donc au maximum

$$\begin{aligned} & O(D(n) + t(T)) + ((D(n) + 1) + 2\,m(T)) - (t(T) + 2\,m(T)) \\ &= O(D(n)) + O(t(T)) - t(T) \\ &= O(D(n)) , \end{aligned}$$

puisque l'on peut r valuer les unit s de potentiel de fa on qu'elles dominent la constante cach e dans $O(t(T))$. Intuitivement, le co t de chaque liaison est compens par la diminution de potentiel due au fait que la liaison diminue de un le nombre de racines. On verra la section 20.4 que $D(n) = O(\lg n)$, de sorte que le co t amorti de l'extraction du nœud minimal est $O(\lg n)$.

Exercices

20.2.1 Montrer le tas de Fibonacci r sultant d'un appel EXTRAIRE-MIN-TAS-FIB sur le tas de Fibonacci de la figure 20.3(m).

20.2.2 D montrer que le lemme 19.1 est valide pour les arbres binomiaux non tri s, si l'on remplace la propri t 4 par la propri t 4′.

20.2.3 Montrer que, si les op rations de tas fusionnable sont les seules support es, le degr maximal $D(n)$ d'un tas de Fibonacci n nœuds vaut au plus $\lfloor \lg n \rfloor$.

20.2.4 Le professeur Blaise a labor une nouvelle structure de donn es bas e sur les tas de Fibonacci. Un tas de Blaise poss de la m me structure qu'un tas de Fibonacci et supporte les op rations de tas fusionnable. Les impl mentations des op rations sont les m mes que pour les tas de Fibonacci, hormis que l'insertion et l'union font une consolidation comme derni re phase de leur travail. Quels sont les temps d'ex cution des op rations de tas de Blaise, dans le cas le plus d favorable ? La structure de donn es du professeur est-elle vraiment in dite ?

20.2.5 Montrer que, quand les seules op rations admises sur les cl s sont des comparaisons entre deux cl s (comme c'est le cas pour toutes les impl mentations propos es dans ce chapitre), il est impossible que toutes les op rations de tas fusionnable puissent s'ex cuter en temps amorti $O(1)$.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

20.3 DIMINUTION D'UNE CL ET SUPPRESSION D'UN NŒUD

Dans cette section, nous allons voir comment de diminuer la cl d'un nœud dans un tas de Fibonacci dans un temps amorti $O(1)$ et comment supprimer un nœud d'un tas de Fibonacci n nœuds dans un temps amorti $O(D(n))$. Ces op rations ne pr - servent pas la propri t selon laquelle tous les arbres d'un tas de Fibonacci sont des arbres binomiaux non tri s. N anmoins, ils en sont assez proches pour qu'on puisse borner le degr maximal $D(n)$ par $O(\lg n)$. Prouver l'existence de cette borne, chose que sera faite la section 20.4, permet d'inf rer que EXTRAIRE-MIN-TAS-FIB et SUPPRIMER-TAS-FIB s'ex cutent dans un temps amorti $O(\lg n)$.

a) Diminution d'une cl

Dans le pseudo code suivant qui impl mente l'op ration DIMINUER-CL -TAS-FIB, on suppose comme pr c demment que supprimer un nœud d'une liste cha n e ne modifie aucun des champs contenus dans le nœud supprim .

```
DIMINUER-CL -TAS-FIB(T, x, k)
1  si k > cl [x]
2     alors erreur « nouvelle cl plus grande que cl courante »
3  cl [x] ← k
4  y ← p[x]
5  si y ≠ NIL et cl [x] < cl [y]
6     alors COUPER(T, x, y)
7           COUPE-EN-CASCADE(T, y)
8  si cl [x] < cl [min[T]]
9     alors min[T] ← x

COUPER(T, x, y)
1  supprimer x de liste des enfant de y, en d cr mentant degr [y]
2  ajouter x liste de racines de T
3  p[x] ← NIL
4  marqu [x] ← FAUX

COUPE-EN-CASCADE(T, y)
1  z ← p[y]
2  si z ≠ NIL
3     alors si marqu [y] = FAUX
4           alors marqu [y] ← VRAI
5           sinon COUPER(T, y, z)
6                 COUPE-EN-CASCADE(T, z)
```

La proc dure DIMINUER-CL -TAS-FIB fonctionne de la mani re suivante. Les lignes 1–3 assurent que la nouvelle cl n'est pas plus grande que la cl courante de x, puis assignent la nouvelle cl x. Si x est une racine, ou si $cl\ [x] \geqslant cl\ [y]$, alors y est

Scanned with CamScanner

le parent de x et aucune modification de la structure n'est n cessaire, puisque l'ordre de tas min a t respect . Les lignes 4–5 testent cette condition.

Si l'ordre de tas min n'a pas t respect , de nombreux changements peuvent survenir. On commence par ***d tacher x*** la ligne 6. La proc dure COUPER « coupe » le lien entre x et son parent y, ce qui fait de x une racine.

On utilise les champs *marqu* pour obtenir les bornes temporelles d sir es. Ces champs enregistrent une petite partie de l'historique de chaque nœud. Supposons que x soit un nœud qui a v cu les v nements suivants :

1) une poque, x tait une racine,
2) ensuite, x a t reli un autre nœud,
3) enfin, deux enfant de x ont t supprim s via des coupes.

D s qu'il a perdu son deuxi me enfant, x est coup de son parent, ce qui en fait une nouvelle racine. Le champ *marqu* $[x]$ a la valeur VRAI si les tapes 1 et 2 ont eu lieu et si un enfant de x a t coup . La proc dure COUPER efface donc *marqu* $[x]$ la ligne 4, puisqu'elle ex cute l' tape 1. (On peut voir pr sent pourquoi la ligne 3 de RELIER-TAS-FIB efface *marqu* $[y]$: le nœud y est en train d' tre reli un autre nœud et l' tape 2 est donc en cours. La prochaine fois qu'un enfant de y sera coup , *marqu* $[y]$ prendra la valeur VRAI.)

Nous n'avons pas encore termin , car x pourrait tre le deuxi me enfant coup de son parent y depuis la derni re fois o y a t reli un autre nœud. La ligne 7 de DIMINUER-CL -TAS-FIB effectue donc une ***coupe en cascade*** sur y. Si y est une racine, alors le test la ligne 2 de COUPE-EN-CASCADE permet la proc dure de rendre la main imm diatement. Si y n'est pas marqu , la proc dure le marque la ligne 4, puisque son premier enfant vient d' tre coup , puis rend la main. Toutefois, si y est marqu , c'est qu'il vient de perdre son deuxi me enfant ; y est coup la ligne 5 et COUPE-EN-CASCADE s'appelle elle-m me r cursivement la ligne 6 avec comme param tre z, le parent de y. La proc dure COUPE-EN-CASCADE remonte ainsi dans l'arbre, jusqu' trouver soit une racine, soit un nœud non marqu .

Une fois que toutes les coupes en cascade ont eu lieu, les lignes 8–9 de DIMINUER-CL -TAS-FIB terminent le traitement en mettant jour *min*$[T]$ si n cessaire. Le seul nœud dont la cl a chang tait le nœud x dont on a diminu la cl . Par cons quent, le nouveau nœud minimal est soit le nœud minimal originel soit le nœud x.

La figure 20.4 montre l'ex cution de deux appels DIMINUER-CL -TAS-FIB, partir du tas de Fibonacci repr sent la figure 20.4(a). Le premier appel, montr la figure 20.4(b), ne requiert aucune coupe en cascade. Le deuxi me appel, montr aux figures 20.4(c)–(e), induit deux coupes en cascade.

Nous allons montrer pr sent que le co t amorti de DIMINUER-CL -TAS-FIB vaut seulement $O(1)$. On commence par d terminer son co t r el. La proc dure DIMINUER-CL -TAS-FIB prend un temps $O(1)$, plus le temps d'ex cution des

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

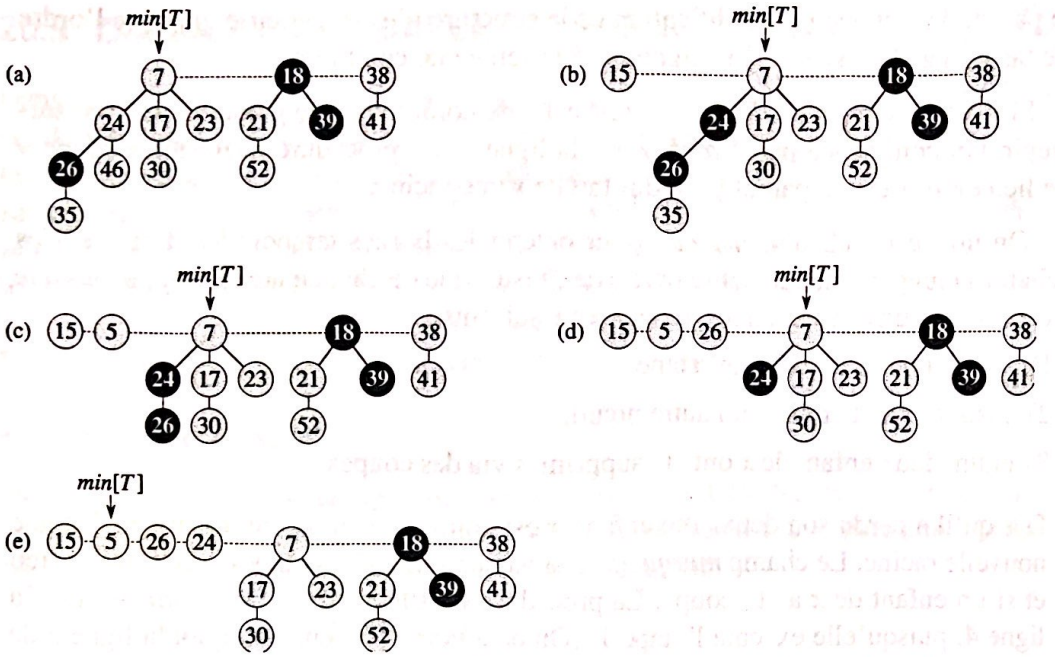


Figure 20.4 Deux appels DIMINUER-CL -TAS-FIB. **(a)** Le tas de Fibonacci initial. **(b)** Le nœud de cl 46 voit sa cl prendre la valeur 15. Le nœud devient une racine et son parent (de cl 24), qui avait pr c demment perdu sa marque, est marqu nouveau. **(c)–(e)** Le nœud de cl 35 voit sa cl diminuer pour prendre la valeur 5. Dans la partie (c), le nœud, contenant pr sent la cl 5, devient racine. Son parent, le nœud de cl 26, est marqu , ce qui engendre une coupe en cascade. Le nœud de cl 26 est coup de son parent et devient une racine non marqu e (d). Une autre coupe survient, puisque le nœud de cl 24 est galement marqu . Ce nœud est coup de son parent, et devient une racine non marqu e dans la partie (e). Les coupes en cascade s'arr tent cet endroit, puisque le nœud de cl 7 est une racine. (M me si le nœud n'avait pas t une racine, les coupes se seraient arr t es, car il est non marqu .) Le r sultat de l'op ration DIMINUER-CL -TAS-FIB est montr dans la partie (e), o *min*[*T*] pointe sur le nouveau nœud minimal.

coupes en cascade. Supposons que COUPE-EN-CASCADE soit appel e r cursivement c fois partir d'un certain appel DIMINUER-CL -TAS-FIB. Chaque appel COUPE-EN-CASCADE prend $O(1)$ si l'on ne compte pas les appels r cursifs. Le co t r el de DIMINUER-CL -TAS-FIB, en prenant en compte tous les appels r cursifs, est donc $O(c)$.

On calcule ensuite la modification du potentiel. Soit T le tas de Fibonacci juste avant l'op ration DIMINUER-CL -TAS-FIB. Chaque appel r cursif COUPE-EN-CASCADE, except le dernier, coupe un nœud marqu et remet z ro le bit de marquage. Apr s l'appel, on est en pr sence de $a(T) + c$ arbres (les $a(T)$ arbres initiaux, $c - 1$ arbres produits par les coupes en cascades et l'arbre de racine x) et au plus $m(T) - c + 2$ nœud marqu s ($c - 1$ ont perdu leurs marques pendant les coupes en cascades et le dernier appel COUPE-EN-CASCADE peut avoir marqu un nœud). Le changement de potentiel est donc au plus

$$((a(T) + c) + 2(m(T) - c + 2)) - (a(T) + 2\,m(T)) = 4 - c\,.$$

Scanned with CamScanner

Donc, le co t amorti de DIMINUER-CL -TAS-FIB vaut au plus

$$O(c) + 4 - c = O(1) ,$$

puisqu'on peut r valuer les unit s de potentiel pour dominer la constante cach e dans $O(c)$.

On peut maintenant voir pourquoi la d finition de la fonction potentiel inclut un terme qui vaut deux fois le nombre de nœuds marqu s. Lorsqu'un nœud y marqu est coup lors d'une coupe en cascade, son bit de marquage est remis z ro et le potentiel est r duit de 2 unit s. Une unit de potentiel compense la coupe et l'effacement du marquage, et l'autre unit compense l'accroissement de potentiel d la transformation du nœud y en racine.

b) Suppression d'un nœud

La suppression d'un nœud d'un tas de Fibonacci n nœuds est facile effectuer dans un temps amorti $O(D(n))$, comme on le voit dans le pseudo code suivant. On suppose qu'il n'existe pas de valeur de cl gale $-\infty$ dans le tas de Fibonacci.

```
SUPPRIMER-TAS-FIB(T, x)
1  DIMINUER-CL -TAS-FIB(T, x, −∞)
2  EXTRAIRE-MIN-TAS-FIB(T)
```

SUPPRIMER-TAS-FIB est analogue SUPPRIMER-TAS-BINOMIAL. Elle fait de x le nœud minimal du tas de Fibonacci en lui donnant une valeur de cl plus petite que toutes les autres, savoir $-\infty$. Le nœud x est alors supprim du tas de Fibonacci par la proc dure EXTRAIRE-MIN-TAS-FIB. Le temps amorti pour SUPPRIMER-TAS-FIB est la somme du temps amorti $O(1)$ de DIMINUER-CL -TAS-FIB et du temps amorti $O(D(n))$ de EXTRAIRE-MIN-TAS-FIB. Comme on verra la section 20.4 que $D(n) = O(\lg n)$, le temps amorti de SUPPRIMER-TAS-FIB est $O(\lg n)$.

Exercices

20.3.1 On suppose qu'une racine x d'un tas de Fibonacci est marqu e. Expliquer comment x a pu devenir une racine marqu e. Montrer qu'il importe peu pour l'analyse que x soit marqu , m me si ce n'est pas une racine qui avait t reli e un autre nœud avant de perdre un enfant.

20.3.2 Justifier le temps amorti $O(1)$ de DIMINUER-CL -TAS-FIB comme co t moyen par op ration, en utilisant l'analyse par agr gat.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

20.4 BORNE POUR LE DEGR MAXIMAL

Pour d montrer que le temps amorti de EXTRAIRE-MIN-TAS-FIB et TAS-FIB-SUPPRIMER est $O(\lg n)$, on doit montrer que la borne sup rieure $D(n)$ du degr d'un nœud d'un tas de Fibonacci n nœuds vaut $O(\lg n)$. D'apr s l'exercice 20.2.3, quand tous les arbres du tas de Fibonacci sont des arbres binomiaux non tri s, $D(n) = \lfloor \lg n \rfloor$. Toutefois, les coupes qui surviennent dans DIMINUER-CL -TAS-FIB, peuvent obliger les arbres contenus dans le tas de Fibonacci ne plus respecter les propri t s d'arbre binomial non tri . Dans cette section, nous allons montrer que le fait de couper un nœud de son parent d s qu'il perd deux enfant implique que $D(n) = O(\lg n)$. Plus pr cis ment, nous d montrerons que $D(n) \leqslant \lfloor \log_\phi n \rfloor$, o $\phi = (1 + \sqrt{5})/2$.

Le principe de l'analyse est le suivant. Pour chaque nœud x du tas de Fibonacci, on d finit taille(x) comme tant le nombre de nœuds, y compris x lui-m me, du sous-arbre enracin en x. (Notez que x n'a pas besoin de faire partie de la liste de racines ; il peut tre n'importe quel nœud.) Nous allons montrer que taille(x) est exponentielle par rapport *degr* [x]. Il faut garder l'esprit que *degr* [x] est constamment maintenu jour en tant que degr de x.

Lemme 20.1 *Soit x un nœud d'un tas de Fibonacci, avec degr* [x] = k. *Soient $y_1, y_2, \ldots, y_k$ les enfant de x dans l'ordre dans lequel ils ont t reli s x, en commen ant par le plus ancien. Dans ce cas, degr* [y_1] $\geqslant 0$ *et degr* [y_i] $\geqslant i - 2$ *pour* $i = 2, 3, \ldots, k$.

D monstration : Manifestement, *degr* [y_1] $\geqslant 0$. Pour $i \geqslant 2$, on remarque qu'au moment o y_i a t reli x, tous les $y_1, y_2, \ldots, y_{i-1}$ taient des enfants de x ; on avait donc forc ment *degr* [x] = $i-1$. Le nœud y_i tant reli x seulement si *degr* [x] = *degr* [y_i], on avait donc aussi *degr* [y_i] = $i - 1$ ce moment-l . Depuis lors, y_i a perdu au plus un enfant, puisqu'il aurait t coup de x s'il en avait perdu deux. On en conclut que *degr* [y_i] $\geqslant i - 2$. ❑

Nous arrivons enfin la partie de l'analyse qui justifie le nom de « tas de Fibonacci ». On se souvient qu' la section 3.2, pour $k = 0, 1, 2, \ldots$, le k me nombre de Fibonacci est d fini par la r currence

$$F_k = \begin{cases} 0 & \text{si } k = 0, \\ 1 & \text{si } k = 1, \\ F_{k-1} + F_{k-2} & \text{si } k \geqslant 2. \end{cases}$$

Le lemme suivant donne un autre moyen d'exprimer F_k.

Lemme 20.2 *Pour tous les entiers $k \geqslant 0$,*

$$F_{k+2} = 1 + \sum_{i=0}^{k} F_i .$$

Scanned with CamScanner

D monstration : La d monstration s'effectue par r currence sur k. Quand $k = 0$,

$$\begin{aligned} 1 + \sum_{i=0}^{0} F_i &= 1 + F_0 \\ &= 1 + 0 \\ &= 1 \\ &= F_2 . \end{aligned}$$

On admet maintenant l'hypoth se de r currence $F_{k+1} = 1 + \sum_{i=0}^{k-1} F_i$, et l'on a

$$\begin{aligned} F_{k+2} &= F_k + F_{k+1} \\ &= F_k + \left(1 + \sum_{i=0}^{k-1} F_i \right) \\ &= 1 + \sum_{i=0}^{k} F_i . \end{aligned}$$

❑

Le lemme suivant et son corollaire compl tent l'analyse. Ils font appel l'in galit (d montr e l'exercice 3.2.7)

$$F_{k+2} \geqslant \phi^k ,$$

o ϕ est le nombre d'or, d fini dans l' quation (3.22) par $\phi = (1+\sqrt{5})/2 = 1{,}61803\ldots$

Lemme 20.3 *Soit x un nœud d'un tas de Fibonacci, avec $k = degr\ [x]$. Alors,* taille$(x) \geqslant F_{k+2} \geqslant \phi^k$, *o* $\phi = (1 + \sqrt{5})/2$.

D monstration : Soit s_k la valeur minimale de taille(z) sur tous les nœuds z tels que $degr\ [z] = k$. Trivialement, $s_0 = 1$, $s_1 = 2$ et $s_2 = 3$. Le nombre s_k vaut au plus taille(x), et visiblement, la valeur de s_k cro t de fa on monotone avec k. Comme pour le lemme 20.1, soient $y_1, y_2, \ldots, y_k$ les enfant de x, dans l'ordre dans lequel ils ont t reli s x. Pour calculer un majorant pour taille(x), on compte un pour x lui-m me et un pour le premier enfant y_1 (pour lequel taille$(y_1) \geqslant 1$), ce qui donne

$$\begin{aligned} \text{taille}(x) &\geqslant s_k \\ &= 2 + \sum_{i=2}^{k} s_{degr\ [y_i]} \\ &\geqslant 2 + \sum_{i=2}^{k} s_{i-2} , \end{aligned}$$

La derni re ligne d coule du lemme 20.1 (qui fait que $degr\ [y_i] \geqslant i - 2$) et de la monotonie de s_k (qui fait que $s_{degr\ [y_i]} \geqslant s_{i-2}$). Nous montrons ensuite par r currence sur k que $s_k \geqslant F_{k+2}$ pour tout entier k positif ou nul. Les bases, pour $k = 0$ et $k = 1$, sont triviales. Pour l' tape inductive, on suppose que $k \geqslant 2$ et que $s_i \geqslant F_{i+2}$ pour $i = 0, 1, \ldots, k - 1$.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner

$$s_k \geqslant 2 + \sum_{i=2}^{k} s_{i-2} \geqslant 2 + \sum_{i=2}^{k} F_i = 1 + \sum_{i=0}^{k} F_i = F_{k+2} \quad \text{(d'apr s le lemme 20.2)} .$$

Nous avons donc d montr que taille$(x) \geqslant s_k \geqslant F_{k+2} \geqslant \phi^k$. ❑

Corollaire 20.4 *Le degr maximal $D(n)$ d'un nœud d'un tas de Fibonacci n nœuds est $O(\lg n)$.*

D monstration : Soit x un nœud d'un tas de Fibonacci n nœuds, et soit $k = degr\,[x]$. D'apr s le lemme 20.3, on a $n \geqslant$ taille$(x) \geqslant \phi^k$. En prenant les logarithmes de base ϕ, on obtient $k \leqslant \log_\phi n$. (En fait, comme k est un entier, $k \leqslant \lfloor \log_\phi n \rfloor$.) Le degr maximal $D(n)$ d'un nœud vaut donc $O(\lg n)$. ❑

Exercices

20.4.1 Le professeur Pinocchio affirme que la hauteur d'un tas de Fibonacci n nœuds est $O(\lg n)$. Montrer que le professeur se trompe en trouvant, pour un entier positif n quelconque, une s quence d'op rations de tas de Fibonacci qui cr e un tas de Fibonacci constitu d'un seul arbre qui est une cha ne lin aire de n nœuds.

20.4.2 Supposons qu'on g n ralise la r gle de la coupe en cascade, de telle fa on qu'un nœud x soit coup de son parent d s lors qu'il perd son k me enfant, pour une certaine constante enti re k. (La r gle de la section 20.3 utilise $k = 2$.) Pour quelles valeurs de k a-t-on $D(n) = O(\lg n)$?

PROBL MES

20.1. Autre impl mentation de la suppression

Le professeur Pisano a propos la variante suivante pour la proc dure SUPPRIMER-TAS-FIB, en affirmant qu'elle s'ex cute plus vite lorsque le nœud en cours de suppression n'est pas le nœud point par $min[T]$.

```
SUPPRIMER-PISANO(T, x)
1  si x = min[T]
2     alors EXTRAIRE-MIN-TAS-FIB(T)
3     sinon y ← p[x]
4           si y ≠ NIL
5              alors COUPER(T, x, y)
6                    COUPE-EN-CASCADE(T, y)
7           ajouter liste des enfant de x  liste de racines de T
8           supprimer x de la liste de racines de T
```

Scanned with CamScanner

a. L'affirmation du professeur selon laquelle cette proc dure s'ex cute plus vite est bas e en partie sur l'hypoth se que la ligne 7 peut tre ex cut e dans un temps r el $O(1)$. O est la faille dans cette hypoth se ?

b. Donner une bon majorant pour le temps r el d'ex cution de SUPPRIMER-PISANO quand $x \neq min[T]$. Votre borne devra tre fonction de $degr\ [x]$ et du nombre c d'appels COUPE-EN-CASCADE.

c. Soit T' le tas de Fibonacci r sultant d'une ex cution de SUPPRIMER-PISANO(T, x). En supposant que le nœud x n'est pas une racine, borner le potentiel de T' en fonction de $degr\ [x]$, c, $a(T)$ et $m(T)$.

d. Conclure que le temps amorti de SUPPRIMER-PISANO n'est asymptotiquement pas meilleur que celui de SUPPRIMER-TAS-FIB, m me quand $x \neq min[T]$.

20.2. D'autres op rations sur les tas de Fibonacci

On souhaite tendre un tas de Fibonacci T pour lui permettre de supporter deux nouvelles op rations sans changer le temps d'ex cution amorti des autres op rations.

a. Donner une impl mentation efficace de TAS-FIB-MODIFIER-CL (T, x, k), proc dure qui donne la cl du nœud x la valeur k. Analyser le temps d'ex cution amorti de votre impl mentation selon que k est sup rieur, inf rieur ou gal $cl\ [x]$.

b. Donner une impl mentation efficace de ELAGUER-TAS-FIB(T, r), qui supprime $\min(r, n[T])$ nœuds de T. Les nœuds supprimer devront tre arbitraires. Analyser le temps amorti d'ex cution de votre impl mentation (*Conseil :* On pourra avoir besoin de modifier la structure de donn es et la fonction potentiel.)

NOTES

Les tas de Fibonacci on t introduits par Fredman et Tarjan [98]. Leur article d crit galement l'application des tas de Fibonacci aux probl mes des plus courts chemins origine unique, des plus courts chemins pour toutes paires, du couplage bipartie pond r et de l'arbre couvrant minimal.

Ensuite, Driscoll, Gabow, Shrairman et Tarjan [81] ont d velopp « les tas rel ch s » comme alternative aux tas de Fibonacci. Il existe deux vari t s de tas rel ch s. L'une donne les m mes bornes temporelles amorties que les tas de Fibonacci. L'autre permet DIMINUER-CL de s'ex cuter en temps (non amorti) $O(1)$ dans le cas le plus d favorable, et EXTRAIRE-MIN et SUPPRIMER de s'ex cuter en temps $O(\lg n)$ dans le cas le plus d favorable. Les tas rel ch s sont galement plus avantageux que les tas de Fibonacci pour les algorithmes parall les.

Reportez-vous au notes du chapitre 6 pour conna tre d'autres structures de donn es qui permettent des op rations DIMINUER-CL rapides quand la suite des valeurs retourn es par des appels EXTRAIRE-MIN cro t de fa on monotone avec le temps et que les donn es sont des entiers appartenant un intervalle sp cifique.

© Dunod – La photocopie non autoris e est un d lit

Scanned with CamScanner